

DSA STUDY BLUEPRINT SERIES

103. Inorder Predecessor Successor in BST

Finding BST Inorder Predecessor and Successor Nodes

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Successor:** Smallest element strictly greater than target key.
- **Predecessor:** Largest element strictly smaller than target key.

Memory & Execution Blueprint

Search key: 5 • Node value 6 > 5 $\Rightarrow$ Update successor candidate to 6 • Move Left

C++ Implementation Reference

Standard code layout and syntax

```
TreeNode* getSuccessor(TreeNode* root, int key) {  
    TreeNode* succ = nullptr;  
    while (root) {  
        if (root->val > key) { succ = root; root = root->left; }  
        else root = root->right;  
    }  
    return succ;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(H)$
Space Complexity	$O(1)$

Key Practices & Gotchas

- **Important:** This method avoids full tree traversals, resolving nodes in $O(H)$ time.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

Active Node	Valid range limit	Comparison check	Recursive branch step
15 (Root)	$[-\infty, +\infty]$	True $(-\infty < 15 < +\infty)$	Recurse Left child with range $[-\infty, 15]$
10 (Left)	$[-\infty, 15]$	True $(-\infty < 10 < 15)$	Recurse Right child with range $[10, 15]$

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.